GhostBusters Phase 2: Final Evidence Pack

Read-only source audit and presentation verification • Prepared 30 July 2026 • Evidence is limited to executable code, configuration, tests, fixtures, UI and safe runtime checks.

> **Verified positioning.** GhostBusters is a safety-first, evidence-aware cloud-cost investigation and Terraform PR-review platform. Its complete demo flow is fixture-backed: parse Terraform input → choose allowlisted evidence tools → create alternatives → calculate confidence/conflicts/verifier/policy → mandatory human review → simulated PR by default. A real GitHub remediation PR is optional and configuration-dependent. No automatic approval, merge, Terraform apply, cloud deployment or cloud-resource mutation was found.

1. Runtime sanity check

Check	Verified result / evidence
Project root	`C:\Users\varsh\Desktop\GhostBusters`.
Python	Runtime command `python --version` returned **Python 3.14.4**. The repository does not pin a Python version.
Virtual environment	`run_demo.ps1` and `test_demo.ps1` expect `.venv\Scripts\Activate.ps1`; no `.venv` was available in this audit environment.
Install/start command	`python -m pip install -r requirements.txt`; then `uvicorn app.main:app --reload` (also in `run_demo.ps1`).
Default URL / access	`http://127.0.0.1:8000`, from `Settings.app_base_url`. Register through `POST /api/auth/register` or UI; login is available.
Demo mode	Set `DEMO_MODE_ENABLED=true`. It exposes fixture scenarios through `/api/scenarios`; fixture plans are under `fixtures/terraform`, scenarios under `fixtures/scenarios`.
Database/Redis/Docker	PostgreSQL and Redis are optional in development, but production configuration validation requires both. `docker-compose.yml` provides PostgreSQL 17 and Redis 7; Docker Compose is not needed for the fixture/in-memory demo.
Required settings	Development works with defaults after dependencies install. Production requires `AUTH_REQUIRED`, secret, database/Redis URLs, CORS, proxy settings; real integrations require their respective credentials/configuration.
Safe runtime result	`python -c/import app.main` failed because `fastapi` is missing. `python -m pytest -q` failed because `pytest` is missing. No packages were installed, per audit constraint.

Commands run: `python --version` succeeded; FastAPI import failed: `ModuleNotFoundError: No module named 'fastapi'`; app import failed for the same reason; pytest failed: `No module named pytest`. Therefore no pass/fail/skip count is available. `DEMO_GUIDE.md` says “install dependencies” but omits the exact pip command; `run_demo.ps1` is outdated for a fresh machine because it assumes an already-created venv.

2. Main working user flows

Flow	Trigger / UI / API	Processing and output	Mode / readiness / limitation
Fixture Terraform review	PR Reviews; `POST /api/runs`.	`WorkflowService.start_run` parses fixture plan, plans, investigates, evaluates and pauses for review.	Fixture-backed; strongest demo path.
Goal validation/clarification	Goals “Review Goal”; `POST /api/goals/validate`.	Validates scope/safety; returns questions/suggested goal.	Implemented; deterministic unless optional AI production path.
Connected goal	Goals “Start Investigation”; `POST /api/goals`.	`start_connected_goal` selects connected read-only collectors; records evidence or abstains/escalates.	Partial; not full goal-to-remediation automation.
Cloud Hunt	Cloud Hunt; `POST /api/cloud/hunts`.	`CloudHuntService` scans normalized inventory and opens review cases.	Fixture-backed by default; AWS real mode exists.
GitHub PR webhook	GitHub webhook; `POST /webhooks/github`.	Signature/allowlist validation, PR/files read, Terraform diff parse, run creation.	Optional real GitHub integration.
Human review	Approvals; `POST /api/runs/{id}/review`.	Approve, reject, modify, request evidence, add context, revoke/reopen. Version/idempotency checked.	Implemented and demo-ready.
PR result	Approval result.	`create_mock_pull_request` emits simulated PR; optional `RemediationPRService` creates real branch/file/PR.	Simulated default; real requires explicit config.

Most reliable presentation flow: safe fixture review → approval → simulated PR, then conflicting or missing-evidence scenario. It is self-contained and visibly labels fixture/simulated data.

3. Three-scenario reasoning comparison

Reasoning stage	Safe	Conflicting evidence	Missing evidence	Evidence source
Input/resource	Staging EC2 right-sizing.	Terraform fixture designed for contradictory Jira/Git activity.	Terraform fixture designed with unavailable evidence.	`fixtures/scenarios/*.json`, `fixtures/terraform/*.json`
Environment/change	Staging; instance type change.	Fixture-defined Terraform change.	Fixture-defined Terraform change.	Scenario + plan parser
Planner/tools	Pricing, utilization, Jira, git activity, dependencies as relevant.	Contextual tools selected to expose conflict.	Evidence tools return unavailable records.	`core/planner.py`, `core/investigator.py`
Utilization	**18% average, 31% peak, 14 days**.	Fixture-defined; inspect scenario at demo time.	Unavailable/limited by fixture.	`safe.json`; mock tool records
Pricing	**$140 current, $70 proposed/month**.	Fixture-defined.	Unavailable or insufficient.	`safe.json`; pricing tool
Business/repository/dependencies	Jira FINOPS-101 completed; 0 recent commits, 42 days; no active dependencies.	Jira completion conflicts with recent activity.	Evidence gaps prevent confidence.	Scenario fixtures, mock Jira/git/dependency tools
Conflicts/missing evidence	No intended conflict; complete evidence.	Conflict detector flags inconsistent evidence.	Missing-evidence records trigger escalation.	`conflict_detector.py`, `investigator.py`
Alternatives	Downsize/schedule/keep/request-evidence/abstain as eligible.	Safer evidence request preference.	Request evidence or abstain/keep.	`alternative_generator.py`
Savings	**$70/month; $840/year** estimate.	Do not present unverified scenario values.	Do not present savings.	`tests/test_alternatives.py`
Confidence/policy	Calculated from complete fixture evidence; must be shown from runtime record, not preclaimed.	Conflict penalty reduces confidence.	Critical missing evidence blocks remediation.	`confidence.py`, policy/Conftest
Final handling	Pending human review; approval produces simulated PR.	Escalation/request more evidence.	Safe escalation/abstention.	`WorkflowService`

All displayed safe-scenario numbers are fixture-backed estimates, not production measurements. The conflicting scenario is the strongest proof of non-static reasoning: contradictory business and repository evidence changes the preferred action away from straightforward optimization; missing evidence separately prevents a recommendation.

4. Autonomous reasoning evidence

Capability	Status and evidence	Honest judge wording
Goal, validation, vague-goal clarification, measurable suggestion	Implemented: `/api/goals/validate`, deterministic ambiguity checks, `GoalValidationResponse`.	“It turns a bounded cloud-cost goal into a safe investigation scope and asks when essential details are missing.”
Plan and dynamic tool selection	Partial: `AIPlanner.plan`, `AgentNextAction`, validator and allowlist; deterministic planner remains authority.	“The model can choose the next registered read-only evidence tool, but policy and tool permissions are not model-controlled.”
Three-plus external tools	Implemented paths: GitHub, AWS and Jira; optional credentials required.	“Three real integration paths are implemented; the fixture demo does not require them.”
Alternatives / preferred action	Implemented: `alternative_generator.py`, `_select_preferred`.	“It compares downsize, schedule, keep, request-evidence and abstain/blocked outcomes.”
Conflict, missing evidence, abstention, blocking	Implemented: detector, missing records, policy, final statuses.	“Incomplete or contradictory evidence decreases confidence and routes to humans rather than forcing a change.”
Retry/fallback/stop safely	Implemented: `RetryExecutor`; classified retries; unavailable evidence/safe failure.	“Transient tools are retried; exhausted or unsafe evidence stops the workflow safely.”

Explanation/audit	Implemented: evidence, decisions, audit events, optional AI explanation.	“It exposes evidence, checks, score components and recorded actions—not hidden chain-of-thought.”

Decision ownership: AI: objective interpretation, optional goal validation/plan, next-tool proposal, explanation. Deterministic: parsing, tool allowlisting, alternatives, conflicts, confidence, verifier and retry. Policy: Rego/Conftest or Python fallback. Conditional logic: production/destructive gates and action ordering. Human: all approval/rejection/waiver actions.

> **Judge-ready answer:** GhostBusters is a bounded hybrid agent. It can accept a goal and optionally use an AI model to select the next allowlisted read-only evidence action, but deterministic verification, policy checks and human approval remain the decision authority. This is deliberate enterprise autonomy: it investigates independently and stops before mutation.

5. Actual components and agents

Active component	Path/function	Role / status
AIPlanner / agent loop	`core/ai_planner.py: AIPlanner`; `core/agent_loop.py: run_agent_investigation`	Optional AI next-action loop; active when configured; AI-driven, bounded.
Goal AI planner	`app/main.py` goal creation route	Optional allowlisted capability plan; production-configured path; partial.
Deterministic planner/investigator	`core/planner.py:create_investigation_plan`; `core/investigator.py:collect_evidence`	Active deterministic planning/collection.
Reasoning components	`alternative_generator.py`, `conflict_detector.py`, `confidence.py`, `verifier.py`, `policy_engine.py`, `reasoning_engine.py`	Active deterministic/policy decision pipeline.
Workflow/PR/review/outcome	`workflow_service.py`, `remediation_pr_service.py`, `human_review.py`, `outcome_verification.py`	Active; real PR path optional; outcome verification is partial/manual.
Cloud Hunt	`cloud_hunt_service.py`	Active, primarily fixture-backed inventory path.

Search did not find real active classes named Goal Planner Agent, Repository Understanding Agent, Cloud Hunt Agent, Optimizer Agent, Recommendation Ranking Agent, PR Review Agent, PR Approval Agent, LangGraph, or an Investigator–Optimizer–Verifier multi-agent system. **Presentation architecture sentence:** “GhostBusters combines one bounded AI planning loop with deterministic evidence, safety and approval services, rather than claiming independent specialist-agent collaboration.”

6. External tools and integrations

Integration	Capability / auth / mode	Safety/default
GitHub REST, App, webhook	`github_client.py`, `github_app.py`, `github_webhook.py`; token/App installation and webhook secret.	Reads PR/files/reviews; optional guarded writes; not enabled by default.
AWS STS, EC2, EBS, EIP, Pricing, CloudWatch	`RealAWSCloudAdapter`; default/assumed role with external ID.	Read-only; optional; unavailable evidence on failure.
Jira	`jira_client.py`; base URL/email/API token.	Read-only optional.
Terraform CLI	`TerraformRunner`.	Disabled by default; version/fmt-check/validate/show JSON only.
OPA/Conftest	Rego policy plus optional CLI.	Python fallback when unavailable.
PostgreSQL/Redis/Docker/Render	Compose/schema/render config.	Postgres/Redis optional locally, required production; Docker/Render deployment tooling.
Gemini/Groq	`ai_client.py`; API keys.	Optional; structured schema and deterministic fallback. Groq dependency gap noted.
Cost Explorer, Linear, GitLab, OpenTofu, OpenAI, Slack, Teams	No implementation found.	Absent.
Azure/GCP	Fixture adapters only.	Not real integrations.

Count: 3 genuinely implemented external integration paths (GitHub, AWS, Jira), plus optional model providers and infrastructure tooling. Show GitHub/AWS/Jira as implemented but optional; label Azure/GCP fixture-only and all absent integrations as absent.

7. Human-in-the-loop and safety

Human approval is mandatory at the `pending_human_review` state before remediation. AI cannot approve recommendations or PRs. The application has no GitHub merge call, Terraform apply invocation, deploy route, terminate call, or direct cloud mutation. A real branch, file commit and PR can occur only if `GITHUB_CREATE_REAL_PR=true`, GitHub integration/token are configured, a human approves, and `RemediationPRService.create` passes its guards. Simulated PR is the default.

Safety condition	Code/policy evidence
Production/destructive/replacement	`planner.py` skips normal optimization; `ghostbusters.rego` denies; real PR service rejects destructive/replacement and production.
Dependencies/unknown ownership/critical evidence/low confidence	Rego denial rules; policy threshold default **0.70**; verifier/policy fallback.
Conflicts/API failure/ambiguous edit	Conflict penalty/requests evidence; `RetryExecutor`; exact single replacement match required.
Human controls	Approve, reject, modify, request evidence, add context, revoke/reopen; waiver in Cloud Hunt. Permission, idempotency, expected-version, actor/audit recording.

> **Safety statement:** “GhostBusters can investigate and prepare a narrowly scoped remediation, but production-risk policy and a mandatory authenticated human decision stop it before any infrastructure mutation.”
> **Why no automatic change?** “Cloud evidence can be incomplete or stale, and business context belongs to accountable owners.”
> **Why approval is a feature:** “It creates segregation of duties, traceability and accountable change control required by enterprise operations.”

8. Explainability and decision trail

Feature	Frontend-visible	API / file
Live plan/current stage/tool attempts	Yes: Goals workspace, Live Plan and Technical Trace.	`static/index.html`, `static/app.js`; goal/run APIs.
Evidence, freshness, reliability and contexts	Yes when recorded.	Run decision/evidence and `/api/goals/{id}/events`.
Alternatives, savings, confidence, risk, policy, verifier	Yes in review/detail panels.	`DecisionRecord`, `reasoning_engine.py`.
Missing/conflicting evidence and failures	Yes.	`investigator.py`, audit/activity APIs.
Approval/reviewer/activity/technical audit	Yes, permission-dependent.	`/api/runs/{id}/review`, `/api/activity`.
Outcome verification	Available through outcome controls.	`/api/outcomes`, `outcome_verification.py`.

User-facing explanation is structured evidence/check/score data. Audit events are persisted records. Backend logs are developer logs. Optional Gemini explanation is constrained to recorded facts in `gemini_explanation.py`. Do not describe hidden model chain-of-thought. Emphasize the Live Plan, Evidence, confidence/policy, approval state and Technical Audit.

9. Presentation screen-capture plan

Screen	Navigation/scenario	Show / label / criterion
Overview	Overview after safe run.	Attention, savings opportunities, recent review, timeline; label demo data.
Goal validation	Goals; enter ambiguous “15%” goal.	Clarification controls; autonomous reasoning.
Live plan/evidence	Goals workspace or PR detail; safe.	Tools, evidence, limitations; transparent reasoning.
Alternatives/confidence/policy	Safe review detail.	Options, confidence, verifier/policy result; architecture/safety.
Approval/simulated PR	Approvals; approve safe.	Human boundary then explicitly labelled simulated PR.
Conflict/missing evidence	Run `conflicting` / `missing_evidence`.	Changed decision and safe stop.
Activity/technical audit	Activity Log / Technical Audit.	Recorded event trail; hide emails, repository names and tokens.

Cloud Hunt	Cloud Hunt fixture.	Inventory candidates; label fixture-backed and Azure/GCP fixture-only.

Branding assets: `static/assets/Ghostopslogo.png`, `static/assets/ghostops-sidebar.png`, `static/assets/ghostops-favicon.png`. No verified screenshots/architecture diagram assets were found. Crop UI to workflow data and hide account IDs, emails, repository names or any local browser details.

10. Metrics and presentation-safe numbers

Metric	Value / source	Safe wording
Safe scenario current/proposed cost	$140 / $70 monthly, `fixtures/scenarios/safe.json`.	“Fixture demo estimate.”
Estimated savings	$70/month, $840/year, `safe.json`, `tests/test_alternatives.py`.	“Fixture-backed estimate.”
Utilization	18% average, 31% peak, 14-day window, `safe.json`.	“Fixture utilization evidence.”
Policy confidence	0.70 default, `app/settings.py`.	“Minimum remediation confidence threshold.”
Cloud Hunt thresholds	0.45 candidate; 0.75 high confidence; CPU threshold 10%, `settings.py`.	“Configured thresholds.”
Fixture inventory	10 resources: AWS 4, Azure 3, GCP 3, `cloud_inventory.json`.	“10 fixture resources.”
Human action count	7 workflow action types; Cloud Hunt adds waiver, based on route/action maps.	“Permissioned review actions including approve, reject and request evidence.”

Do not use: savings percentages, accuracy, coverage, customer adoption, realized savings, response time, scenario time, PR-generation time or test pass count—none was verified.

11. Old slide claim audit

Claim	Verdict / corrected wording
“30% of cloud resources are wasted”; “demo proves 20%+ savings”	**FALSE/UNVERIFIED.** No such measured result. Use labelled fixture estimates only.
“Project-aware AI agent”; “understands code, cloud and business intent”	**PARTIALLY TRUE.** Bounded read-only GitHub/Jira/AWS context exists; not broad repository understanding.
“Continuously monitors infrastructure”	**PARTIALLY TRUE.** Scheduling exists; real monitoring is optional AWS collection, demo is fixture-based.
“Creates review-ready PRs”	**PARTIALLY TRUE.** Simulated by default; real guarded GitHub PR optional.
Auto-approve/merge/deploy	**FALSE.** No implementation; human approval is mandatory and no apply/merge/deploy exists.
Infrastructure graph / multiple specialized agents / fully autonomous	**FALSE.** No graph or independent specialist-agent system; bounded hybrid workflow.
“Dynamically chooses tools/adapts to evidence”	**PARTIALLY TRUE.** Bounded tool choice inside deterministic allowlists/policy.
“Supports AWS, Azure and GCP”	**PARTIALLY TRUE.** Real AWS optional; Azure/GCP fixture-only.
“Integrates GitHub, Jira and AWS”	**TRUE, optional.** Real code paths require configuration/credentials.
GitLab/Linear/Slack/Teams; Gemini/OpenAI/Claude/Groq	**FALSE.** Gemini and optional Groq only; others absent.
React/TypeScript/Tailwind	**FALSE.** Vanilla HTML/CSS/JS.
FastAPI; PostgreSQL/Redis; OPA/Conftest	**TRUE/PARTIAL.** FastAPI actual; Postgres/Redis optional dev/required production; OPA/Conftest optional with fallback.
“Learns continuously”	**FALSE.** Outcome records exist but no learning loop.
“Verifies outcomes after deployment”	**PARTIALLY TRUE.** Manual confirmation/observation workflow; no deployment execution.
Safe failures/abstention/blocks production/destruction	**TRUE.** Retry, policy and final-state code support it.

12. Phase 2 judging mapping

Criterion	Show / emphasize	Weakness / avoid	Requirement status
B2B Impact 25%	Auditable FinOps recommendation and controlled change review.	No customer or realized-savings evidence.	Partially satisfied

Autonomous Reasoning 25%	Goal clarification, bounded next-tool selection, alternatives, conflict/abstention.	Do not call fully autonomous or multi-agent.	Partially satisfied
Architecture 20%	FastAPI, services, policies, audit, optional integrations.	Real integrations not runtime-verified here; AWS-only real cloud.	Strongly/partially satisfied
Human-in-loop 15%	Approval screen, policy block, activity trail.	None—this is strongest area.	Strongly satisfied
Live demo 15%	Three fixture scenarios and simulated PR, transparently labelled.	External live path needs preflight.	Strongly satisfied for fixtures

13. Likely judge questions: concise answers

Question	Spoken answer	Technical backup
What is autonomous?	“It independently plans a bounded evidence investigation, selects allowlisted read-only tools, compares actions and stops safely; approval remains human.”	`AIPlanner` optional; deterministic planner/policy authority.
Is it multi-agent?	“No—not independent specialist agents. It is a bounded hybrid of one optional AI planning loop and deterministic safety services.”	No LangGraph/specialist classes found.
How differs from Cost Explorer/Infracost?	“It combines Terraform/PR context, business evidence, dependency/policy checks and human approval rather than only reporting cost.”	Cost Explorer itself is absent; do not compare as an integration.
Why AI?	“AI helps interpret scoped goals and choose the next registered evidence question; it does not control policy or infrastructure.”	Structured model output is validated/allowlisted.
What if AI is wrong?	“Invalid tool/action proposals are rejected, deterministic validation remains authoritative, and evidence gaps route to a human.”	`ai_plan_validator.py`, policy and review.
Can it shut down/deploy/merge?	“No. It never applies Terraform, deploys, terminates resources or merges PRs.”	No corresponding calls/routes found.
Does it create a real PR?	“The demo creates a simulated PR. A real GitHub PR is possible only after a human approval and explicit configuration.”	`GITHUB_CREATE_REAL_PR=false` default.
Is demo real AWS?	“The reliable demo is fixture-backed. A separate optional AWS read-only adapter exists.”	Fixtures clearly identified; adapter uses boto3.
Azure/GCP/Jira/three tools?	“Azure/GCP are fixtures only; Jira is optional real read-only. GitHub, AWS and Jira are three implemented external paths.”	Integration modules listed above.
Conflict/missing evidence/confidence?	“Conflicts apply confidence penalties; critical missing evidence blocks remediation and requests more evidence or abstains.”	Detector, confidence calculator, Rego/Python policy.
Does it learn?	“It stores outcome-verification observations, but it does not yet use them to retrain or automatically alter ranking.”	`outcome_verification.py`.
Strongest business value?	“It shortens safe investigation and review by assembling evidence, alternatives and controls into one auditable decision record.”	Do not quantify unmeasured time savings.

14. Presenter’s software walkthrough

Reliable 3-minute walkthrough

1. **0:00—PR Reviews:** start the `safe` scenario. Say: “This is fixture-backed demonstration data; the workflow is the product behaviour we are validating.”
2. **0:30—Plan and evidence:** show parsed Terraform, selected evidence and 14-day CPU/pricing context. Say: “The system gathers only registered evidence sources and records what is unavailable.”
3. **1:15—Alternatives/confidence/policy:** show downsize/keep/request-evidence, confidence, verifier and policy. Say: “The model may help select the next question, but these safety checks are deterministic.”
4. **2:00—Approval:** approve. Say: “Here is the enterprise boundary: human approval is mandatory.”
5. **2:20—Simulated PR:** show it. Say: “This is explicitly simulated; real GitHub PR creation is separately enabled and still never merges or deploys.”
6. **2:40—Conflict or missing evidence:** show one. Say: “Changing evidence changes the outcome; it escalates instead of forcing a recommendation.”

Stronger 5-minute version

Use the 3-minute flow, then show an ambiguous 15% goal validation question, the Technical Audit timeline, and the missing-evidence case. Emphasize B2B control, autonomous bounded planning, architecture, human oversight and demo

transparency.

Backup 2-minute version

Open an existing safe case; show evidence → policy/confidence → approval → simulated PR. Do not depend on live AWS, GitHub or AI provider.

Emergency wording: If startup/slow API fails, present the recorded fixture backup and say it is the verified deterministic path. If AI/GitHub/AWS is unavailable, say optional integrations fail safely and do not silently substitute live claims. If simulated PR is misunderstood: “No external call was made in this demo.” If asked for deployment: “It intentionally has no deployment capability.”

15. Final presentation fact pack

- **Product:** GhostBusters is a safety-first platform for evidence-aware cloud-cost investigations and Terraform PR reviews.
- **Problem:** Cloud-cost changes require technical, utilization, business and policy context; scattered evidence makes safe review slow.
- **Differentiator:** It turns evidence into alternatives, confidence, policy checks and an auditable human decision—not an opaque recommendation.
- **Autonomy:** A bounded hybrid: optional AI selects next allowlisted read-only investigations; deterministic systems and humans control action.
- **Safety/human control:** Production/destructive/low-confidence conditions block remediation, and human approval is mandatory.
- **Architecture:** FastAPI + static UI + deterministic reasoning/policy/audit services + optional GitHub/AWS/Jira/model integrations.
- **Live demo:** Fixture-safe recommendation, simulated PR, then conflict/missing-evidence escalation.
- **Three capabilities:** evidence/policy reasoning; human-controlled remediation review; visible audit trail.
- **Three benefits:** safer FinOps decisions; faster evidence assembly; enterprise accountability.
- **Safe values:** 0.70 policy confidence threshold; 10 fixture inventory resources; safe fixture estimate $70/month/$840/year (label it).
- **Limitations:** fixture default, incomplete connected-goal remediation, no apply/merge/deploy or real Azure/GCP.
- **Avoid:** automatic deployment/approval/merge; Cost Explorer; multi-agent collaboration; real multi-cloud monitoring; verified customer savings.
- **Verified stack:** Python/FastAPI/Pydantic/Uvicorn, vanilla HTML/CSS/JS, optional PostgreSQL/Redis, boto3, Gemini, Docker Compose.
- **Verified integrations:** optional GitHub, AWS, Jira; Azure/GCP fixtures only.
- **Roadmap, not current:** Cost Explorer, real Azure/GCP, graph, learning loop, deploy/merge automation.
- **Simulated PR:** an in-app deterministic PR representation; it makes no GitHub call.
- **Optional real PR:** guarded GitHub branch/file/PR creation after authenticated human approval and explicit configuration.
- **What GhostBusters is today:** a credible, transparent safety-first investigation and review platform—not an autonomous infrastructure deployment system.

> **Final quality check passed:** this pack does not claim automatic approval, merge, Terraform apply or deployment; labels fixture data and simulated PRs; treats real GitHub PR creation as optional/configuration-dependent; treats Azure/GCP as fixture-only; does not claim Cost Explorer, React/TypeScript/Tailwind, OpenAI/Claude/Slack/Teams/Linear/GitLab, or independent specialist agents.